While there is no industry standard yet regarding how to represent profiles on the wire or at rest (say, as a file in your file system), the `pprof` format is the closest we have to such a standard, and many of the tools available can read or generate it.

`pprof` as a format is highly efficient, as it's based on Protocol Buffers, but that also means it can be hard to work with compared to a setup where you get JSON from an HTTP API (and can use `curl` and `jq` to filter for any value). There is some tooling available to deal with `protobuf`, such as

- *Protoc (https://grpc.io/docs/protoc-installation/)*—Short for *protocol buffer compiler*, which you can use to compile .proto files but also to decode or deserialize them (useful for debugging)
- *Buf (https://github.com/bufbuild/buf)*—A powerful CLI tool for working with protobuf
- *Protoman (https://github.com/spluxx/Protoman)*—A Postman-like API client for protobuf-based messages
- *Postman (https://github.com/postmanlabs/postman-app-support/issues/2801#issuecomment-1002247567)*—A popular tool that has added (indirect) support for it via a new gRPC API type

What does debugging a protobuf file look like?

Let's use `protoc` for that. First, we need to get a `pprof` protobuf formatted file. We, indeed, already produced a couple of those in our previous example, where we used `go tool pprof` for profiling. Note that your setup may have a different path configured; look at the output of the `go tool pprof` command, where it says Saved Profile In … (mine is in ~/pprof). Execute the following in the `ch09/cp/` directory:

```
$ cp ~/pprof/pprof.samples.cpu.001.pb.gz \
     ./pprof.gz && gzip -d pprof.gz
```

Now that we have the `pprof` protobuf file, we can use `protoc` to deserialize it:

This is the pprof protobuf-encoded file we created in the previous step and that protoc is supposed to use as input.

We tell protoc to use pprof.proto as the schema and ask it to decode a specific message: perftools.profiles.Function.

```
$ protoc --decode=perftools.profiles.Function pprof.proto \
      < pprof | \
      grep --color=auto -e ".*Task"

6: "main.slowTask"
```

Once protoc is done, we search for all function symbols that contain "Task".

As expected, we found at least one function mentioned in the pprof file, and that happens to be slowTask() in my case.

With that, we're done for now with `pprof`, but we will certainly see it again in action later in the chapter.